Traces (Distributed)

What is a Trace?

> A trace is the full journey of ONE request through your system.

Think of it like a **delivery tracking number** 📦

Pizza analogy 🍕

You order a pizza:

1. Order received
2. Pizza prepared
3. Baked
4. Packed
5. Delivered

A **trace shows all these steps**, how long each took, and where it failed.

In software terms

When a user hits your app:

```
Request ID: abc-123

→ API Gateway (10ms)
→ Auth Service (15ms)
→ Orders Service (120ms) ❗ slow
→ Database (300ms) ❗ very slow
→ Response
```

👉 That **entire path = ONE TRACE**

Trace vs Logs vs Metrics

Beginners often confuse these. Let's clear it forever.

Type	Answers what?	Example
Logs	What happened?	“Order created”
Metrics	How often / how much?	CPU = 80%
Traces	Why is THIS request slow or broken?	Order → DB took 300ms

Simple rule:

- Logs → **Details**
- Metrics → **Trends**
- Traces → **Root cause**

You need **all three** for real systems.

Why Traces are needed

Without traces ❌:

- “App is slow”
- “Which service?”
- “Which DB call?”
- “Which user request?”

With traces ✅:

- “Order API is slow”
- “Because DB query in Orders service”
- “Only for POST /order”
- “Latency = 300ms”

🔥 That’s **power**

How tracing works (conceptually)

Step-by-step (don’t worry, no code yet)

1. User sends a request
2. System creates a **Trace ID**
3. That Trace ID is passed to:
    - Service A
    - Service B
    - Service C
4. Each service records:
    - Start time
    - End time
    - Errors
5. All data is sent to a **Tracing backend**

👉 Result: a **visual timeline**

Key terms you must know

Term	Meaning
Trace	Full request journey
Span	One step in that journey
Trace ID	Unique request ID
Span ID	Unique step ID

Example:

```
Trace: Order Pizza
 ├── Span: API
 ├── Span: Auth
 ├── Span: Orders
 └── Span: DB
```

Deployement

Explanation

Dashboard for Traces